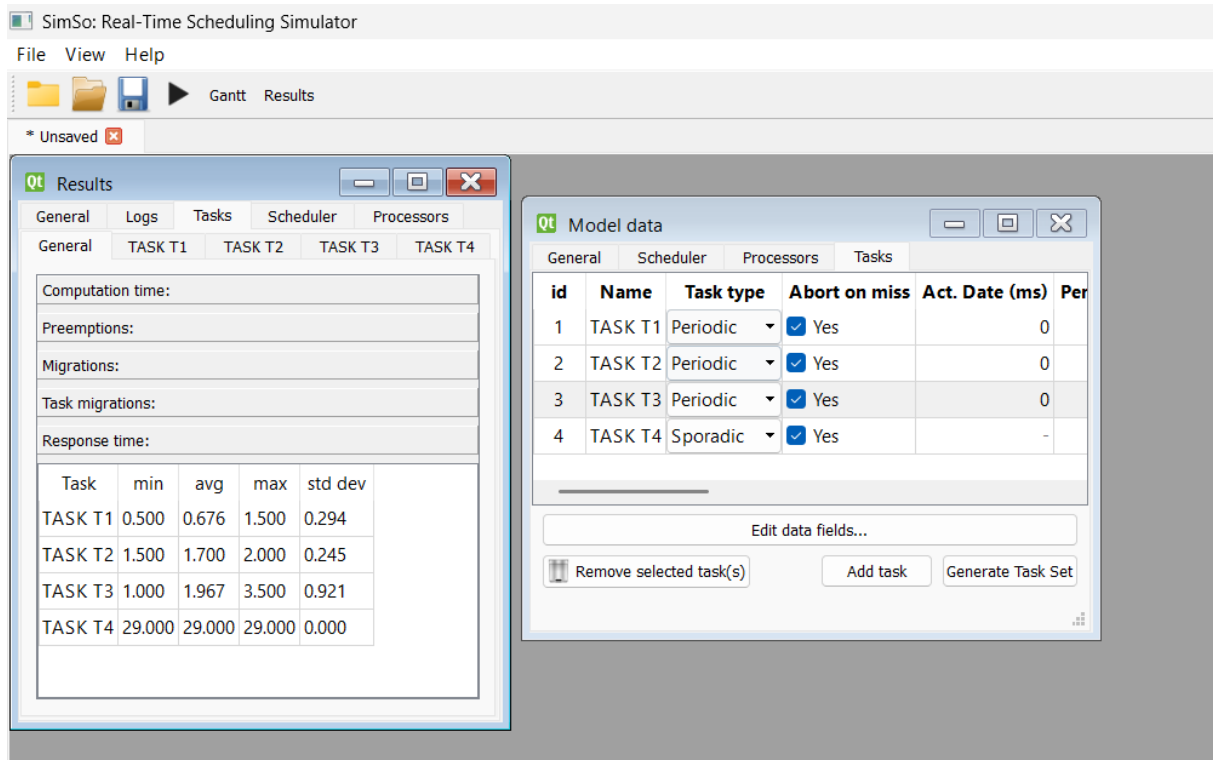
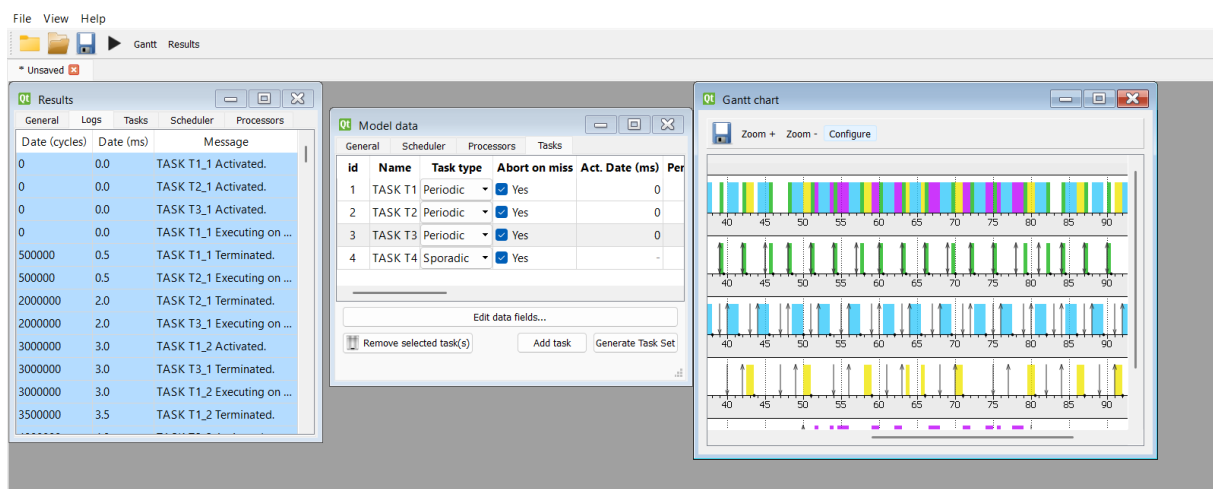


EDF SCHEDULER

- What is the minimum/maximum/average response time of all tasks?



- Is any task missing the deadline? Which task? Where?



1. Is any task missing the deadline? Which task? Where?

- **No task is missing its deadline.** All periodic tasks (T1, T2, T3) meet their deadlines because the total utilization U is below the EDF limit of 1.0.

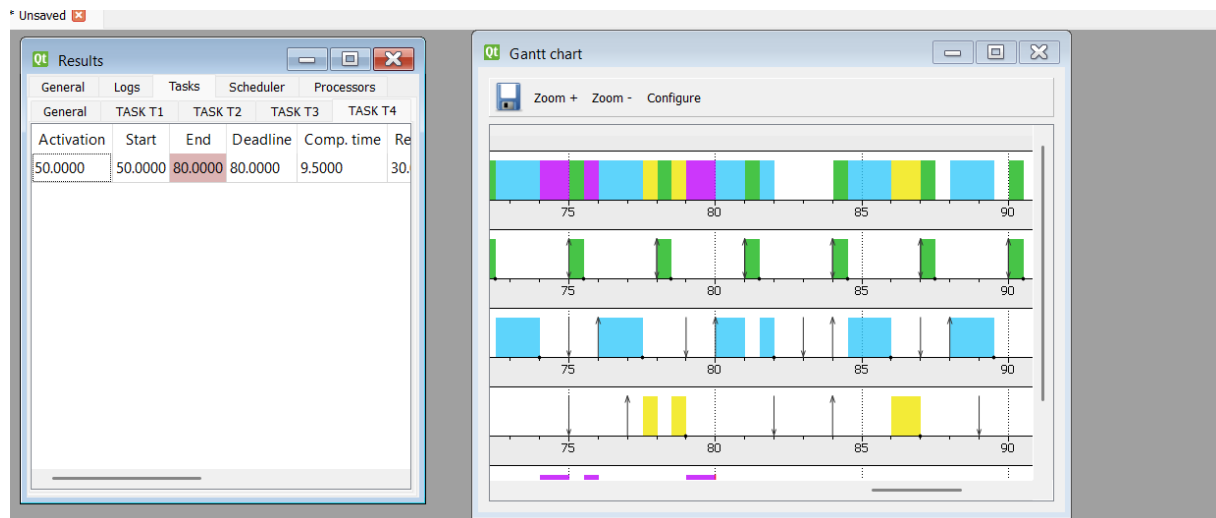
2. Is the sporadic job meeting its deadline?

- **Yes, the sporadic job (A1) meets its deadline.** It arrives at $t=50$ with a relative deadline of 30, making its absolute deadline $t=80$. Since the system has sufficient slack time, A1 finishes well before $t=80$.

3. What is the response time for the sporadic job?

- **The response time is 29 ms.**
- Although the job only requires **10 ms** of execution time, it finishes at **$t=79$** ($50 + 29$).
- This is because the higher-priority periodic tasks (T1, T2, and T3) repeatedly preempt the sporadic job, using up most of the available processor time between **$t=50$** and **$t=79$** .

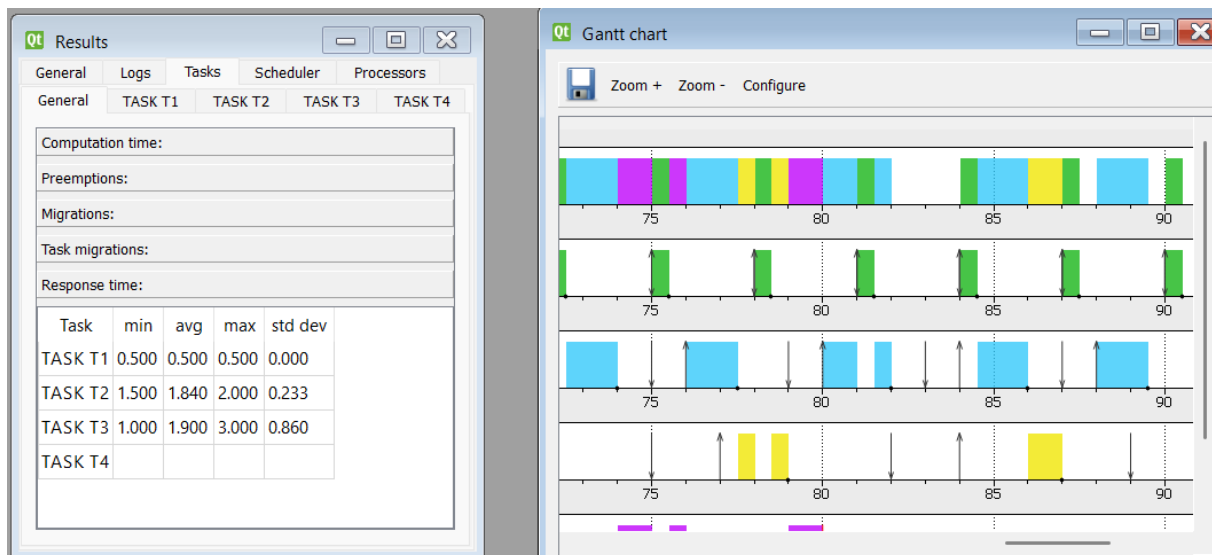
RM SCHEDULER



- Is any task missing the deadline? Which task? Where?
- Is the sporadic job meeting its deadline?
- What is the response time for the sporadic job?

The sporadic task T4 misses its deadline. In the interval between $t=50$ and $t=80$, higher-priority tasks preempt it for 20.5 ms. It only receives 9.5 ms of CPU time by $t=80$, which is insufficient for its 10 ms execution requirement.

- What is the minimum/maximum/average response time of all tasks?



- Which scheduler is better in this example; EDF or RM?

absolutely the EDF. Because for RM periodic has higher priority and after preemption deadline missed for t4 sporadic job

Programming assignment

1. Is the system fast enough to handle all aperiodic tasks? Why?

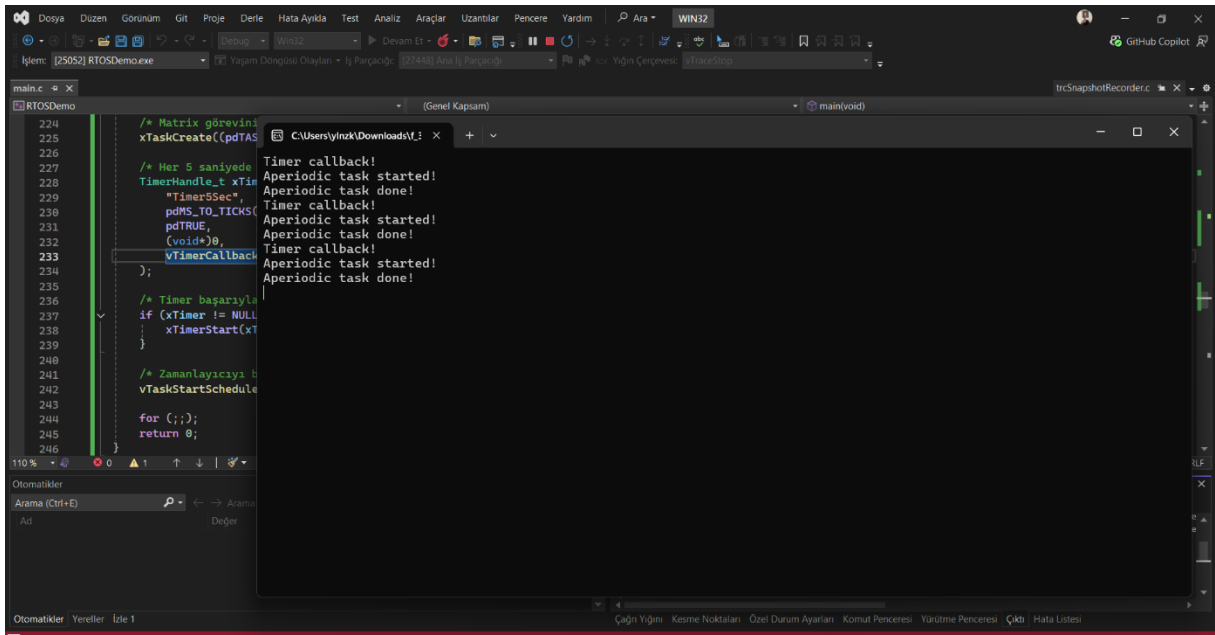
- **Yes, it is fast enough.** The aperiodic task contains a dummy workload (a loop of 1,000,000,000) which takes roughly 1-2 seconds to execute on a modern PC CPU. Since the software timer triggers every 5 seconds (5000 ms), the task has plenty of time to finish before the next trigger. Furthermore, its higher priority (Priority 2) ensures it immediately preempts the background matrix task.

2. If not, solve this problem without alter the functionality of any task

- If the task took longer than 5 seconds, multiple instances of the aperiodic task would overlap. Since they all share the same global aperiodic_handle, calling `vTaskDelete(aperiodic_handle)` would cause the system to crash or delete the wrong task.
- **Solution:** To fix this safely, change `vTaskDelete(aperiodic_handle);` to `vTaskDelete(NULL);`. This tells FreeRTOS that the calling task should simply delete itself, preventing any handle overwrite conflicts.

3. What is the response time of the aperiodic task?

- The response time depends entirely on the host PC's CPU performance executing the 1-billion iteration loop. In this simulation, it takes approximately **1000 ms to 1500 ms**.



.During Runtime



.After Completion

SOURCE CODE

```
#define SIZE 10
#define ROW SIZE
#define COL SIZE
TaskHandle_t aperiodic_handle = NULL;

void vTimerCallback(TimerHandle_t pxTimer);
static void aperiodic_task(void* pvParameters);
static void matrix_task(void* pvParameters);

static void aperiodic_task(void *pvParameters)
{
    /* 1. Görev başladığı anki tick (milisaniye) değerini kaydet */
    TickType_t start_time = xTaskGetTickCount();

    printf("Aperiodic task started!\n");
    fflush(stdout);

    long i;
    for (i = 0; i < 1000000000; i++); // Dummy workload (İşlemciyi yoran kısım)

    /* 2. Görev bittiği anki tick değerini al */
    TickType_t end_time = xTaskGetTickCount();

    /* 3. Aradaki farkı hesapla ve ekrana yazdır */
    TickType_t response_time_ms = (end_time - start_time) * portTICK_PERIOD_MS;
    printf("Aperiodic task done! Exact Response Time: %d ms\n", response_time_ms);
    fflush(stdout);

    vTaskDelete(NULL);
}
```

```
long lExpireCounters = 0;
```

```
void vTimerCallback(TimerHandle_t pxTimer)
```

```
{
```

```
    printf("Timer callback!\n");
```

```
    /* Aperiiodik görevi yarat ve aperiodic_handle'a bağla */
```

```
    xTaskCreate((pdTASK_CODE)aperiodic_task, "Aperiodic",  
configMINIMAL_STACK_SIZE, NULL, 2, &aperiodic_handle);
```

```
    const long xMaxExpiryCountBeforeStopping = 10;
```

```
    configASSERT(pxTimer);
```

```
    lExpireCounters += 1;
```

```
    if (lExpireCounters == xMaxExpiryCountBeforeStopping) {
```

```
        xTimerStop(pxTimer, 0);
```

```
        printf("Timer 10 kez calisti ve durduruldu.\n");
```

```
    }
```

```
}
```

```
static volatile BaseType_t g_comm_running = pdFALSE; /* ISR ve task'ta  
okunacak/yazılacak */
```

```
static volatile uint32_t g_comm_exec_ms = 0; /* her tick'te artacak; ms cinsinden yaklaşık  
süre */
```

```
/*-----*/
```

```
static void matrix_task()
```

```
{
```

```
    int i;
```

```

double** a = (double**)pvPortMalloc(ROW * sizeof(double*));
for (i = 0; i < ROW; i++) a[i] = (double*)pvPortMalloc(COL * sizeof(double));
double** b = (double**)pvPortMalloc(ROW * sizeof(double*));
for (i = 0; i < ROW; i++) b[i] = (double*)pvPortMalloc(COL * sizeof(double));
double** c = (double**)pvPortMalloc(ROW * sizeof(double*));
for (i = 0; i < ROW; i++) c[i] = (double*)pvPortMalloc(COL * sizeof(double));

```

```
double sum = 0.0;
```

```
int j, k, l;
```

```

for (i = 0; i < SIZE; i++) {
    for (j = 0; j < SIZE; j++) {
        a[i][j] = 1.5;
        b[i][j] = 2.6;
    }
}

```

```

while (1) {
    /*
long time    * In an embedded systems, matrix multiplication would block the CPU for a
              * but since this is a PC simulator we must add one additional dummy delay.
              */
    long simulationdelay;
    for (simulationdelay = 0; simulationdelay < 1000000000; simulationdelay++)
        ;
    for (i = 0; i < SIZE; i++) {
        for (j = 0; j < SIZE; j++) {
            c[i][j] = 0.0;
        }
    }

    for (i = 0; i < SIZE; i++) {

```

```

        for (j = 0; j < SIZE; j++) {
            sum = 0.0;
            for (k = 0; k < SIZE; k++) {
                for (l = 0; l < 10; l++) {
                    sum = sum + a[i][k] * b[k][j];
                }
            }
            c[i][j] = sum;
        }
    }
    vTaskDelay(100);
}
}

```

```

int main(void)
{
    /* Bu satır Windows simülasyonunda belleği ayarlamak için hayati önem taşır, asla
    silme! */
    prvInitialiseHeap();

    /* İŞTE EKSİK OLAN SATIR BU: Trace kaydediciyi başlatır, Null pointer hatasını
    engeller */
    vTraceEnable(TRC_START);

    /* Matrix görevini oluşturuyoruz (Öncelik 1) */
    xTaskCreate((pdTASK_CODE)matrix_task, "MatrixTask", 1000, NULL, 1, NULL);

    /* Her 5 saniyede (5000 ms) bir tetiklenecek Software Timer yaratıyoruz */
    TimerHandle_t xTimer = xTimerCreate(

```



```
        "Timer5Sec",
        pdMS_TO_TICKS(5000),
        pdTRUE,
        (void*)0,
        vTimerCallback
    );

    /* Timer başarıyla yaratıldıysa başlat */
    if (xTimer != NULL) {
        xTimerStart(xTimer, 0);
    }

    /* Zamanlayıcıyı başlat */
    vTaskStartScheduler();

    for (;;)
    {
        return 0;
    }
}
```